Connecting Software Reliability Growth Models to Software Defect Tracking

Maskura Nafreen, Melanie Luperon, and Lance Fiondella
Department of Electrical and Computer Engineering
University of Massachusetts Dartmouth
North Dartmouth, MA 02747, USA
Email: {mnafreen, mluperon, lfiondella}@umassd.edu

Vidhyashree Nagaraju
Tandy School of Computer Science
University of Tulsa
Tulsa, OK 74104, USA
Email: vidhyashree-nagaraju@utulsa.edu

Ying Shi
Goddard Space Flight Center
National Aeronautics and Space Administration
Greenbelt, MD 20771, USA
Email: ying.shi@nasa.gov

Thierry Wandji
Cyber Warfare Detachment
Naval Air Systems Command
Patuxent River, MD, 20670, USA
Email: ketchiozo.wandji@navy.mil

***Abstract*— Traditional software reliability growth models only consider defect discovery data, yet the practical concern of software engineers is the removal of these defects. Most attempts to model the relationship between defect discovery and resolution have been restricted to differential equation-based models associated with these two activities. However, defect tracking databases offer a practical source of information on the defect lifecycle suitable for more complete reliability and performance models. This paper explicitly connects software reliability growth models to software defect tracking. Data from a NASA project has been employed to develop differential equation-based models of defect discovery and resolution as well as distributional and Markovian models of defect resolution. The states of the Markov model represent thirteen unique stages of the NASA software defect lifecycle. Both state transition probabilities and transition time distributions are computed from the defect database. Illustrations compare the predictive and computational performance of alternative approaches. The results suggest that the simple distributional approach achieves the best tradeoff between these two performance measures, but that enhanced data collection practices could improve the utility of the more advanced approaches and the inferences they enable.**

***Keywords*—Software reliability, software reliability growth model, software defect lifecycle, defect resolution, Markov modeling**

I. Introduction

Software reliability growth models (SRGM) [1] offer to complement the software testing process in order to quantify the decreasing trend in times between the discovery of defects and the corresponding increase in reliability. Myriad papers call out the limitations of these models such as the assumption made by nonhomogeneous Poisson process (NHPP) SRGM that discovered defects are corrected immediately and no additional defects introduced. The many papers offering incremental advancements overshadow more innovative studies that attempt to establish concrete connections to the activities performed as part of software engineering processes. Despite these efforts, theory and practice have diverged, missing opportunities to drive meaningful advances. Marginal modeling efforts that do not explicitly validate underlying theories and poor data collection practices are two primary impediments. Modelers need to characterize the activities performed by processes and practitioners need to implement controls to capture data more consistently and accurately. The ideal outcome is a virtuous cycle in which modelers equip practitioners with practical inferences of high concern to inform process improvement and practitioners enhance data collection to support the livelihood of modelers.

Early studies that have attempted to characterize software defect discovery and resolution include the work of Schneidewind [2] who modeled defect discovery with a discrete exponential mean value function and resolution with a time lag. Xie and Zhao [3] subsequently extended Schneidewind's model, assuming a defect resolution rate proportional to the number of defects discovered but not yet resolved and also demonstrated the appliability of the Poisson thinning process to model the difference between defects discovered but not yet resolved. Ohba [4] proposed the inflexion S-shaped model to characterize the fact that some defects may need to be discovered and resolved before others can be discovered, while and Yamada and Osaki [5] introduced the delayed S-shaped model, which incorporates a time delay to model this dependence. Similarly, Kapur and Younes [6] modeled leading and dependent defects. Yamada et al. [7] proposed two SRGM with imperfect debugging, where new defects are sometimes introduced when resolving others. Gokhale et al. [8] used a non-homogeneous continuous time Markov chain to model various repair scenarios, analyzing the effect of fault removal policies on the number of defects remaining at the end of the test.

Later studies, include the work of Gokhale and Mullen [9] who developed multi-priority queuing models for the software

2332-6549/20/$31.00 ©2020 IEEE
DOI 10.1109/ISSRE5003.2020.00022

Authorized licensed use limited to: University of Massachusetts - Dartmouth. Downloaded on April 09,2021 at 16:01:02 UTC from IEEE Xplore. Restrictions apply.

defect resolution process, considering the effect of queuing system structures, priority levels, and priority disciplines on the differential mean times to resolution of defects of different severities. Shibata et al. [10] proposed an infinite server queueing model for the defect discovery and resolution process. Huang et al. [11] showed how several existing SRGM can be derived by applying the time-dependent delay function, while Lo and Huang [12] proposed an integrated defect discovery and resolution process modeling framework, in which the defect resolution process is expressed in terms of a time-varying resolution intensity and the difference between the number of defects discovered and resolved. Ullah et al. [13] performed a comparative study of SRGM on discovery and resolution data from dozens of open source software projects. Liu et al. [14] proposed a method to estimate the parameters of a defect removal model from semi-grouped data, which includes approximate times of when defects were discovered and resolved, while Yang et al. [15] model defect detection and correction of multi-release open source software and related optimal release problems. Cinque et al. [16] proposed debugging-workflow-aware SRGM to leverage debugging data managed by companies in bug tracking systems in order to improve accuracy when debugging fails to completely satisfy modelling assumptions. Okamura and Dohi [17] proposed a generalized bivariate fault detection and correction process as well as a model with hyper-Erlang distributions and expectation maximization algorithm to estimate its parameters. Vizarreta et al. [18] found that the inflection S-shaped model best characterized the defect resolution of four releases of an open source software defined network controller and subsequently [19] quantified the cumulative distribution function of time to resolution according to defect severity.

This paper develops defect resolution models based on a data set from a NASA project [20]. Specifically, a differential-equation based model from the framework of Lo and Huang [12] as well as distributional and Markovian models of the defect lifecycle, inspired by the future research recommended in Gokhale and Mullen [9]. These models are created from information within the defect tracking database. The distributional model employs censoring to estimate model parameters when some defects have been discovered but not yet resolved, which is necessary to apply the models and make predictions during the testing and defect resolution process. These methods are applied to answer the following two previously unexplored research questions: (i) Does the Markovian model of defect resolution exhibit greater predictive accuracy than the simpler distribution model of defect resolution? and (ii) Is the defect lifecycle modeled by a Markov chain better characterized by a first or second order Markov chain? That is, do transitions in the defect lifecycle depend only on the present state or does information on the present and prior state determine the next state of the defect lifecycle? To answer (i), we apply the distributional and Markovian models to the data periodically throughout the testing process and compare their predictive accuracies, whereas we employ a formal hypothesis test to answer (ii). Our results indicate that, (i) the Markovian model of defect resolution attains only slightly better predictive accuracy and may therefore not be worth the additional computations. However, the Markovian model provides insight into the defect lifecycle and improved data collection practices may increase its attractiveness. The results also indicate that (ii) a one-step transition probability matrix (first order Markov chain) is sufficient to characterize the transitions in the defect lifecycle. These results are specific to the data set analyzed and results may vary. Moreover, we discuss limitations of the data and the steps we took to conduct our analysis in spite of these deficiencies. Regardless of the results obtained on the data set considered, the methods offer greater insight into the software defect discovery and resolution processes. Methods to improve data quality and that could support enhanced modeling to guide software testing are noted throughout.

Our primary contribution includes joint modeling of defect discovery and resolution that

- Does not assume defect resolution times follow a prescribed parametric form with no explanation in terms of the underlying process associated with defect resolution
- Enables an arbitrary defect lifecycle to be characterized by an a distribution or semi-Markov process
- Can be applied during the defect detection and correction process to track progress

The remainder of the paper is organized as follows: Section II explains the need to distinguish defect discovery and resolution times, while Section III covers defect discovery and resolution models as well as a hypothesis testing method for the order of a Markov chain. Section IV illustrates the defect resolution models. Section V concludes and suggests future research directions.

II. Software Defect Discovery and Resolution: Concepts

For clarity of exposition, Figure 1 shows the less detailed case of defect times data in order to explain many of the core concepts contained in past software defect discovery and resolution models.

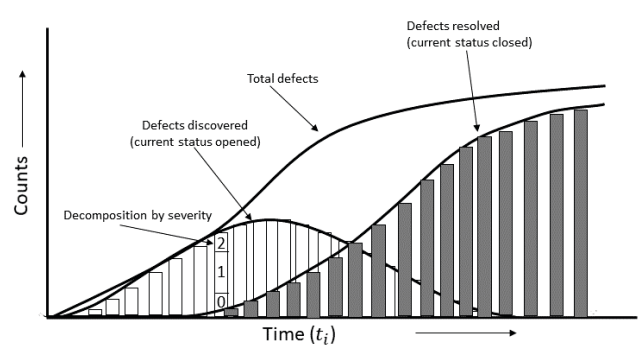

Fig. 1: Software defect discovery and resolution concepts

The upper curve in Figure 1 represents the total defects discovered by the end of the ith interval, which is the cumulative

Authorized licensed use limited to: University of Massachusetts - Dartmouth. Downloaded on April 09,2021 at 16:01:02 UTC from IEEE Xplore. Restrictions apply.

number of defects ever discovered. This corresponds to the counting process $N(t)$ to which software reliability growth models are typically applied. However, basing reliability estimates on the non-discovery of additional defects makes strong implicit assumptions about the scope and completeness of testing. Therefore, a more pragmatic approach also employs the data in the bottom left of the figure with white bars, which represents the number of defects discovered but not yet resolved according to their severity, while the curve in the bottom right of the figure with grey bars represents the cumulative number of defect that have been both discovered and resolved $N_r(t)$. Ideally, the number of unresolved defects goes to zero soon after all defects have been discovered, since the goal of software testing is to both detect and remove defects. Thus, Figure 1 indicates that models that only consider $N(t)$ do not properly distinguish between detection and resolution and are therefore inadequate to assure software reliability. To support more detailed models of defect discovery and resolution a natural resource is the defect tracking database, which documents the lifecycle of individual defects, enabling explicit connections between the software process and activities performed. Toward this end, each model developed in the following section explains how data from the defect tracking database can be transformed in support of parameter estimation.

III. Software defect discovery and resolution: Models

This section describes alternative methods to model the discovery and resolution of software defects recorded in a defect tracking database, including novel models based on the integrated defect discovery and resolution process modeling framework proposed by Lo and Huang [12] as well as distributional and Markovian resolution models developed here. These defect resolution models enable practical inferences of interest to the software practitioner that can be made throughout the software testing process, including the expected time required to remove (i) all defects discovered up to the present time t and (ii) all defects discovered up to the present time t as well as all defects anticipated to be discovered.

A. Integrated defect discovery and resolution processes

In the integrated defect discovery and resolution processes modeling framework [12], the rate of defect discovery and resolution are respectively expressed as differential equations possessing forms

$$\frac{dm(t)}{dt} = \lambda(t)(a - m(t)) \tag{1}$$

and

$$\frac{dm_r(t)}{dt} = \lambda_r(t)(m(t) - m_r(t)) \tag{2}$$

where $m(t)$ ($m_r(t)$) denotes the mean value function of the number of defects discovered (resolved) by time t, $\lambda(t)$ ($\lambda_r(t)$) the defect discovery (resolution) rate, and $a > 0$ the number of defects that would be discovered with infinite testing. Thus, Equation (2) expresses the instantaneous rate of change in defect resolution as the product of the defect resolution intensity multiplied by the defects discovered but not yet resolved by time t.

Lo and Huang [12] derived general forms of the solutions for Equations (1) and (2) with initial conditions $m(t) = 0$ and $m_r(t) = 0$ to produce expressions for the mean value function of defect discovery and resolution. They subsequently applied these general forms to explicitly show that $\lambda(t) = \lambda_r(t) = b$ is the special case where the defect discovery and resolution models are characterized by the Goel-Okumoto [21] and Yamada Delayed S-shaped models [5] respectively. They also derived a novel model with unequal discovery and resolution rates $\lambda(t) = b$ and $\lambda_r(t) = c$. However, additional discovery and resolution models for the inflection S-shaped [4] as well as exponential and Rayleigh [22] as well as Weibull [23] testing effort functions were only specified implicitly in terms of $\lambda(t)$ and $\lambda_r(t)$.

We applied the Goel-Okumoto, Weibull, Yamada Delayed S-shaped, inflection S-shaped, Jelinski-Moranda [24], and Geometric model [25] to a recent NASA data set [20] and found that the inflection S-shaped model best characterized the defect discovery process. Therefore, we derived two explicit forms of $m_r(t)$ with Equation (2), assuming defect discovery is modeled by the inflection S-shaped model, possessing mean value function

$$m(t) = a\frac{1 - e^{-bt}}{1 + ce^{-bt}} \tag{3}$$

where, b is the constant defect discovery rate, c is the inflection parameter

$$c = \frac{1 - r}{r}, \qquad r \in (0, 1]. \tag{4}$$

and r is the inflection rate. As r approaches 1.0, the inflection S-shaped model reduces to $m(t) = a(1 - e^{-bt})$, which is the form of the Goel-Okumoto model.

The first form of the defect resolution model assumes defect resolution intensity $\lambda_r(t) = b$, is the same as parameter b present in Equation (3), producing

$$m_r^b(t) = a\left(1 - e^{-bt} + (1 + c)\log\left(\frac{1 + c}{c + e^{bt}}\right)e^{-bt}\right) \tag{5}$$

The second form of the defect resolution model introduces an additional parameter through $\lambda_r(t) = d$, which implies that the defect resolution intensity is distinct from parameter b in Equation (3), producing

$$\begin{aligned} m_r^d(t) \quad = \quad & \frac{a}{c(b+d)}\Bigg(d\ {}_2F_1\left(1, \frac{b+d}{b}; 2 + \frac{d}{b}; -\frac{1}{ce^{-bt}}\right)e^{bt} \\ & -(b+d)\,{}_2F_1\left(1, \frac{d}{b}; \frac{b+d}{b}; -\frac{1}{ce^{-bt}}\right) \\ & +(b+d)\,{}_2F_1\left(1, \frac{d}{b}; \frac{b+d}{b}; -\frac{1}{c}\right)e^{-dt} \\ & -d\,{}_2F_1\left(1, \frac{b+d}{b}; 2 + \frac{d}{b}; -\frac{1}{c}\right)e^{-dt}\Bigg) \end{aligned} \tag{6}$$

Authorized licensed use limited to: University of Massachusetts - Dartmouth. Downloaded on April 09,2021 at 16:01:02 UTC from IEEE Xplore. Restrictions apply.

where ${}_2F_1(a,b;c,z) = \sum_{k=0}^{\infty} \frac{(a)_k (b)_k}{(c)_k} \frac{z^k}{k!}$ is the hypergeometric function with $(q)_k = q(q+1)\ldots(q+k-1)$ for $k > 0$.

Given a defect database possessing discovery and resolution times, $\mathbf{T} = \langle t_1, t_2, \ldots, t_n \rangle$ and $\mathbf{T}^r = \langle t_1^r, t_2^r, \ldots, t_n^r \rangle$, the defect detection model in Equation (3) is applied with $\mathbf{T}$, while the defect resolution models in Equations (5) and (6) may be applied directly to $\mathbf{T}^r$.

B. Distributional Approach

An alternative and potentially simpler method to model the time to correct defects is to compute the times between discovery and resolution and fit a distribution to these statistics. To make resolution time predictions possible throughout the software testing process, it is necessary to rely on the discovery and resolution times available up until time t. However, at any specific time, the number of defects resolved may be strictly less than the number of defects discovered. This discrepancy requires the use of maximum likelihood estimation techniques for censored data [26]. Specifically, the problem of identifying a distribution to best fit the defect resolution times, when only a subset of the defects discovered has been resolved, may be estimated from the following likelihood function

$$Lik = \prod_{i \in \mathcal{R}}^{k} f(t_{(i)};\theta) \prod_{i \in \mathcal{D}}^{n-k} 1 - F(T - t_i;\theta) \tag{7}$$

where k is the number of defects in the set $\mathcal{R}$ discovered and resolved before time T, $(n-k)$ is the number of defects in the set $\mathcal{D}$ that have been discovered by time T but not yet resolved, and θ is the vector of parameters of the distribution being fit to the data. Moreover, $t_{(i)}$ denotes the time between discovery (t_i) and resolution (t_i^r) of the ith defect such that $t_{(i)} = t_i^r - t_i$, whereas the term $1 - F(T - t_i)$ is the probability that a defect discovered at time t_i has not been resolved by time T.

In practice, Equation (7) can be maximized with multiple alternative distributions and the one achieving the best goodness of fit employed to make predictions. One may then compute the mean of this distribution of best fit, which can be interpreted as the mean time to resolve defects. In this manner, the MVF of the number of defects resolved by time t may be expressed as

$$m_r(t) = m(t - E[T_r]) \tag{8}$$

which translates the mean value function of the defect discovery process to the right by the mean time to resolve defects ($E[T_r]$).

C. Markovian Approach

A Markov chain [27] is a stochastic process characterized by a sequence of states X_t, which take values from the finite set of m possible states. In the context of software defect tracking, the states represent the possible stages of the lifecycle from discovery to resolution. Thus, there is a one-to-one correspondence between the states of the defect lifecycle and the one-step transition probability matrix of the Markov Chain.

The first-order Markov hypothesis states that the present state at time t is conditionally independent of those up to and including time $(t-2)$ and only depends on the state at time $(t-1)$ such that

$$\begin{aligned} &P(X_t = i_0 | X_0 = i_t, ..., X_{t-1} = i_1) \\ &= P(X_t = i_0 | X_{t-1} = i_1) = q_{i_1,i_0}(t) \end{aligned} \tag{9}$$

where $i_t, ...i_0 \in \{1, ..., m\}$. Assuming the state transition probability $q_{i_1,i_0}(t)$ is time-invariant reduces to q_{i_1,i_0}, producing a homogeneous Markov chain.

Considering all combinations of i_1 and i_0, we construct the transition matrix

$$Q = \begin{array}{c} \begin{array}{cccc} 1 & .. & .. & m \end{array} \\ \begin{pmatrix} q_{1,1} & q_{1,2} & .. & q_{1,m} \\ q_{2,1} & .. & .. & .. \\ .. & .. & .. & .. \\ q_{m,1} & .. & .. & q_{m,m} \end{pmatrix} \end{array} \begin{array}{c} \\ 1 \\ . \\ . \\ m \end{array}$$

where rows denote the previous state (X_{t-1}), columns the successor state (X_t), and each row sums to 1.0. In practice, the entries $q_{i,j}$ may be determined from the information contained in the defect tracking database by computing the number of times state i transitioned to state j and dividing by the total number of transitions out of state i. This is possible because each row of the defect tracking database provides distinguishable data on the lifecycle (sequence of state transitions) within the defect tracking model.

To compute the average time between defect discovery and resolution, it is necessary to compute (i) the average number of visits to each state of the defect lifecycle and (ii) the average time spent in each state i before transitioning to state j. This first step is accomplished by solving for the fundamental matrix by setting entry $q_{m,m} = 0$, and computing

$$S = \sum_{k=0}^{\infty} Q^k = (I_m - Q)^{-1} \tag{10}$$

where I_m is the identity matrix of dimension $m \times m$. Entry $s_{i,j}$ is the average number of visits to state j given that defect tracking began in state i. Thus, $s_{1,j}$ is the average number of visits to the jth state of the defect tracking lifecycle. Assuming a unique starting state, this notation can be simplified to s_j without loss of generality.

The second step determines the distribution of each transition from state i to state j ($T_{i,j}$) with information contained in the rows of the defect tracking database. Specifically, the time at which each state was entered is recorded as a timestamp. Therefore, it is possible to sort these times to determine the sequence of one-step transitions taken by the defect from discovery to resolution as well as the time between each of these transitions. Computing these transition time statistics for all rows of the database provides multiple observations for each (i,j) transition. A distribution of best fit is then determined with traditional statistical methods and the expectation of this distribution ($E[T_{i,j}]$) is then computed.

Authorized licensed use limited to: University of Massachusetts - Dartmouth. Downloaded on April 09,2021 at 16:01:02 UTC from IEEE Xplore. Restrictions apply.

The average time from defect discovery to resolution according to the Markovian approach may be expressed as follows

$$E[T_r] = \sum_{i=1}^{m} s_i \sum_{j=1}^{m} q_{i,j} E[T_{i,j}] \tag{11}$$

where the inner sum weights the average time to transition from state i to state j ($E[T_{i,j}]$) by the probability of taking this transition out of state i ($q_{i,j}$), while the outer sum weights these transition times out of state i by the average number of visits to state i (s_i). This alternative value of $E[T_r]$ may be substituted into Equation (8) for the mean number of defects resolved by time t.

D. Higher-order Markov Chains

This section describes higher-order Markov chains, maximum likelihood estimation of transition probabilities, and a hypothesis test to identify the order of the Markov chain.

In some circumstances [28], the present state depends on multiple previous states. When the present state depends on the last k states, a kth-order Markov chain with transition probabilities of the form

$$\begin{aligned} &P(X_t = i_0 | X_0 = i_t, ..., X_{t-1} = i_1) \\ &= P(X_t = i_0 | X_{t-k} = i_k, ..., X_{t-1} = i_1) = q_{i_k, i_0(t)} \end{aligned} \tag{12}$$

is appropriate.

For example, a second-order ($k = 2$) Markov chain composed of $m = 3$ three states is characterized by the transition matrix

$$Q = \begin{array}{cc} & \\ X_{t-2} & X_{t-1} \\ 1 & 1 \\ 2 & 1 \\ 3 & 1 \\ 1 & 2 \\ 2 & 2 \\ 3 & 2 \\ 1 & 3 \\ 2 & 3 \\ 3 & 3 \end{array} \begin{array}{c} X_t \\ \begin{array}{ccc} 1 & 2 & 3 \end{array} \\ \begin{pmatrix} q_{111} & q_{112} & q_{113} \\ q_{211} & q_{212} & q_{213} \\ q_{311} & q_{312} & q_{313} \\ q_{121} & q_{122} & q_{123} \\ q_{221} & q_{222} & q_{223} \\ q_{321} & q_{322} & q_{323} \\ q_{131} & q_{132} & q_{133} \\ q_{231} & q_{232} & q_{233} \\ q_{331} & q_{332} & q_{333} \end{pmatrix} \end{array}$$

Each possible combination of k successive observations of the defect tracking process X is a state of the model such that the number of states is equal to m^k. Moreover, when the order is greater than one, the transition matrix Q contains several elements corresponding to transitions that cannot occur because the state X_{t-1} becomes X_{t-2} at the following time step. Thus, it is impossible to transition from the row where $X_{t-1} = i$, but $X_{t-2} = j \neq i$ in the subsequent state. Since the probability of such transitions is 0, they are omitted from Q, which is written more compactly above than a square matrix possessing structural zeros.

Regardless of the order k, each row of the matrix Q possesses $(m-1)$ independent probabilities, since each row is a probability distribution summing to 1.0 and the mth outcome is completely determined by the others. Therefore, the total number of independent parameters to be estimated is $m^k(m-1)$. Letting $n_{i_k \ldots i_0}$ denote the number of times the subsequence of k states $X_{t-k} = i_k, ..., X_{t-1} = i_1, X_t = i_0$ is observed in the data, the maximum likelihood estimate of the corresponding transition probability $q_{i_k \ldots i_0}$ is

$$\hat{q}_{i_k \ldots i_0} = \frac{n_{i_k \ldots i_0}}{n^*_{i_k \ldots i_1}} \tag{13}$$

where

$$n^*_{i_k \ldots i_1} = \sum_{i_0=1}^{m} n_{i_k \ldots i_0}$$

is the total number of subsequences that begin with $n_{i_k \ldots i_0}$ and end with any state $X_t = i_0$.

It is possible to compute a χ^2 statistic in order to identify the order of a Markov Chain. The χ^2 statistic quantifies how expectations compare to observed data under different model assumptions. The most elementary hypothesis test on the order of a Markov chain [29] poses the following null and alternative hypotheses:

$$\begin{array}{lcll} H_0 & : & m = 1 & \textit{The Markov chain is first order} \\ H_1 & : & m = 2 & \textit{The Markov chain is second order} \end{array}$$

To test the significance of dependence of state i_0 on $q_{i_2 i_1 i_0}$, the χ^2-test can be used to assess the disagreement between $q_{i_2 i_1 i_0}$ and $q_{i_1 i_0}$ as

$$\chi^2 = \sum_{i_2 i_1 i_0} \frac{n^*_{i_2 i_1} (q_{i_2 i_1 i_0} - q_{i_1 i_0})^2}{q_{i_1 i_0}} \tag{14}$$

where

$$n^*_{i_2 i_1} = \sum_{i_0=1}^{m} n_{i_2 i_1 i_0} \tag{15}$$

and degrees of freedom

$$df = (r-1)(c-1) \tag{16}$$

where the rows r and columns c respectively correspond to the unique sequences $q_{i_2 i_1}$ and q_{i_0} taken by defects through the defect lifecycle.

If the null hypothesis is rejected in favor of the alternative hypothesis, then the higher-order transition matrix Q can be expanded to include structural zeros and Equations (10) and (11) of the Markovian approach applied to estimate the mean time to resolve defects with a higher-order Markov chain.

IV. Illustrations

This section illustrates the approaches to defect resolution modeling developed in Section III in order to assess their relative predictive ability and computational efficiency. Section IV-A describes the NASA defect tracking database, steps taken to clean the data, and resulting the Markov model of the software defect tracking process. Section IV-B tests the order of the Markov model, according to the methods described in Section III-D.

Authorized licensed use limited to: University of Massachusetts - Dartmouth. Downloaded on April 09,2021 at 16:01:02 UTC from IEEE Xplore. Restrictions apply.

To emphasize the need for software reliability modeling that can be used during the defect and resolution process, two cases are considered: (i) retrospective analysis, characteristic of many historical defect discovery modeling papers, which used all available data and (ii) online analysis, which periodically updates estimates throughout this process to refine estimates and track progress. Section IV-C presents a comparative analysis of the alternative defect resolution models to the complete data and explains the steps taken to apply the Distributional and Markovian approaches. Section IV-D subsequently presents the corresponding analysis of each model's empirical defect resolution intensity with the complete data. Section IV-E presents online predictive analysis, discussing the tradeoffs between model accuracy and computational efficiency.

A. Data Description, Cleaning, and Modeling

The project database is a rich source of information to model the software defect tracking lifecycle. The database consists of $n = 1933$ rows, each of which corresponds to a defect, including the times at which the defect entered a subset of the 13 possible states listed in Table I.

TABLE I: Defect tracking state descriptions

State	Description	State	Description
0	Created	7	In Work
1	Assigned	8	On Hold
2	Build Integration	9	Ready for Closure
3	Canceled	10	Ready for Test
4	Closed	11	Test Complete
5	Closed with Defect	12	Work Completed
6	In Test		

For each row in the database, the subset of columns with non-null entries indicated the sequence of states taken by that particular defect. Thus, sorting this subset of states in ascending order and computing the difference between successive times respectively determined the sequence of states and transition times. These state transitions provided the feasible one-step transitions within a Markov model of the software defect tracking lifecycle. However, several rows exhibited one or more of the following problems:

- State sequences violated the logical order in which work would be performed, including very rarely observed sequences as well as degenerate sequences that were incomplete.
- Necessary states were not recorded.
- Multiple successive states possessed transition times equal to zero, suggesting that progression of the defect through the lifecycle was not properly measured. The most natural explanation is that software engineers simply entered the same time into the database because the process required documenting the defect was resolved.

Ultimately, 1478 rows were eliminated for the reasons described above, leaving just 455 or approximately 25% of the data for analysis. While this remaining data is by no means complete and may introduce statistical bias, it is a sufficiently large sample to illustrate some possible inferences that would be possible. In doing so, we seek to justify the institutionalization of better data collection practices. For example, the database also contained information on the software build (version), and defect type (defects vs. change requests). However, the reduction in sample size resulting from inconsistent data collection practices and lack of mechanisms to ensure data quality prevented this level of analysis. Some concrete strategies to improve data quality would be to (i) provide formal training on the defect lifecycle tracking software and (ii) include contractual requirements specifying the minimum precision of information recorded.

Figure 2 shows the state transition model of the software defect tracking process after cleaning the dataset.

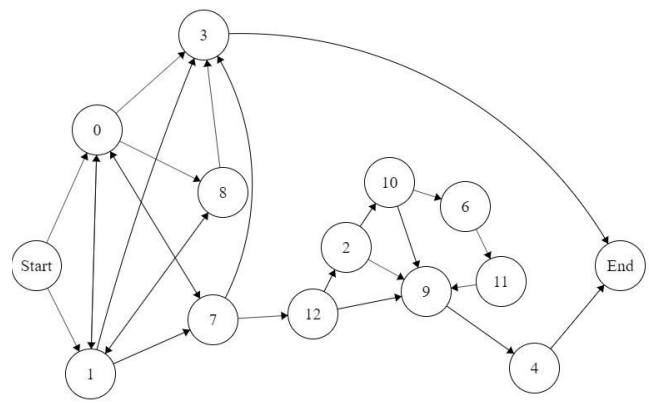

Fig. 2: Software defect tracking state transition model

The software defect tracking state model consists of the 13 states given in Table I and 26 unique one-step transitions, each denoted by a directed edge between two states. Without loss of generality, a defect begins in the *Start* state and completes in the *End* state. These states are placeholders that allow the lifecycle to begin in state *Created (0)* or *Assigned (1)* and end after *Cancelled (3)* or *Closed (4)*. Thus, the transition times out of the *Start* state and into the *End* state are instantaneous, whereas all other transitions possess positive values. Example sequences include (i) *Created (0)* followed by *Assigned (1)* and then *Cancelled (3)* when the issue is determined not to be a defect and (ii) those that enter *In Work (7)*, *Work Completed (12)*, *Ready for Closure (9)*, and *Closed (4)*, possibly also traversing some subsequence of *Build Integration (2)*, *Ready for Test (10)*, *In Test (6)*, and *Test Complete (11)* prior to *Ready for Closure (9)*.

To offer additional insight into the defect tracking process throughout the duration of the project, Figure 3 shows the birth-death process decomposed by defect severity, where defect discovery corresponds to an increase (birth) and defect resolution a decrease (death). Thus, the value on the y-axis corresponds to the difference between the number of defects of a given severity s that were discovered by time t ($N^s(t)$) and the number of defects of the same severity that have been resolved by time t ($N_r^s(t)$), Severity 2 defects are the most serious, while severity 0 defects are the least serious.

Authorized licensed use limited to: University of Massachusetts - Dartmouth. Downloaded on April 09,2021 at 16:01:02 UTC from IEEE Xplore. Restrictions apply.

Medium severity defects were the most abundant ($n_1 = 381$), while fewer low ($n_0 = 61$) and high severity ($n_2 = 13$) defects were documented. At the peak, around $t = 1,200$, over 40 medium severity defects had been discovered but not yet resolved. While the subsequent analysis is not decomposed by severity, Figure 3 provides a window into the complex activities underlying defect tracking and the corresponding opportunities for more detailed modeling.

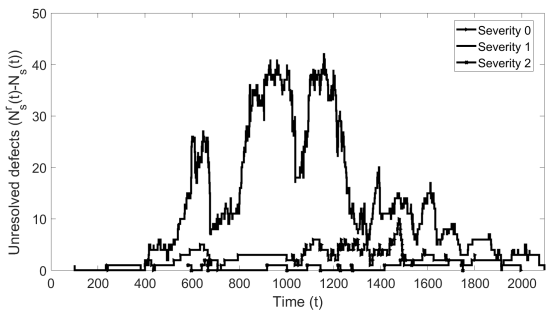

Fig. 3: Defect discovery and resolution by severity

B. Testing the Order of the Markov Chain

This section applies the hypothesis test described in Section III-D to assess a first or second order the Markov chain would best characterize the transitions observed in the sequences of state transitions contained in the rows of the software defect tracking database used to construct Figure 2. Toward this end, Equation (13) was employed to obtain the maximum likelihood estimates of the second order transition probabilities $q_{i_2 i_1 i_0}$ as well as the first order transition probabilities $q_{i_1 i_o}$. These estimates were subsequently substituted into Equation (14) to obtain a χ^2 value of 317.19, which exceeds the critical value 101.88 at the 0.05 level of significance of the χ^2 distribution possessing 80 degrees of freedom, since there were respectively eleven and nine unique second and first order sequences such that $df = (11-1)(9-1)$, according to Equation (16). Thus, the null hypothesis that the Markov chain is first order is rejected at the 0.05 level of significance in favor of the alternative hypothesis that the Markov chain is second order.

The second order transition sequences contributing to the rejection of the null hypothesis were further analyzed to determine if this result possessed an intuitive explanation or if further insight could be made regarding the defect tracking state descriptions given in Table I. To perform this analysis, Table II reports second order transition subsequences contributing to the χ^2 statistic, the numerical value contributed by that term to Equation (14), and the percentage the term contributed to rejection of the null hypothesis. Table II indicates that the transition sequence $1 \to 0 \to 7$ contributed most heavily to rejection of the null hypothesis. However, upon further examination of the meaning of these states in Table I, it was deemed contrary to the logical flow of the defect tracking process. Specifically, it did not seem appropriate to set a defect to *Assigned* before it was *Created*, raising concerns about the consistency of the development teams understanding and use of the defect tracking database. As a result, the subsequent examples apply the Markovian approach described in Section III-C with a first order Markov chain. The primary reasons for this decision are as follows. Although the order of state transitions found at the beginning of the Markov chain may be slightly out of order, (i) this may not significantly alter the performance analysis of the time from *Start* to *End* in Figure 2, (ii) the remainder of the state model contains valuable information on the process underlying defect tracking and corresponding resolution times, and (iii) eliminating these slightly out of order sequences would have substantially reduced the sample size, which would have prevented meaningful application of the Markovian approach. Moreover, despite these caveats related to data quality, the Markovian approach to defect resolution modeling may have merit if defect tracking guidelines are followed more consistently.

TABLE II: Contribution of second order sequences to rejection of the null hypothesis (first order Markov assumption)

Paths	χ^2	Percentage
$0 \to 1 \to 7$	9.36	0.0295
$0 \to 1 \to 8$	0.92	0.0029
$1 \to 0 \to 3$	0.84	0.0026
$1 \to 0 \to 7$	**302.86**	**0.9548**
$1 \to 0 \to 8$	2.91	0.0092
$1 \to 7 \to 3$	0.31	0.0010

C. Defect resolution models

This example examines the accuracy of the alternative defect resolution modeling approaches.

1) Retrospective analysis: Figure 4 shows the defect discovery and resolution counting processes ($N(t)$ and $N_r(t)$) along with the corresponding fitted defect and resolution models as well as distributional and Markovian approaches.

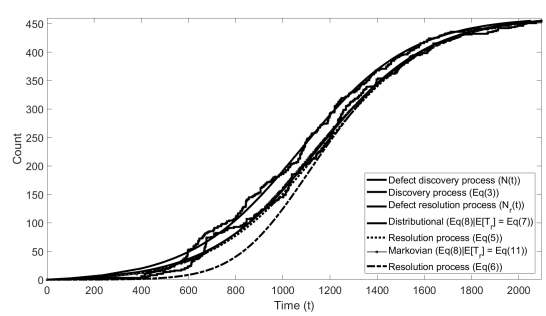

Fig. 4: Defect discovery and resolution processes with fitted defect models

The first counting process (left step function) indicates the times at which the defects were discovered and is accompanied by the fit of the inflexion S-shaped model. The second counting

Authorized licensed use limited to: University of Massachusetts - Dartmouth. Downloaded on April 09,2021 at 16:01:02 UTC from IEEE Xplore. Restrictions apply.

process (right step function) corresponds to the times at which defects were resolved.

For each of the defect resolution modeling approaches, Table III summarizes the sum of squares error (SSE) and runtime in seconds.

TABLE III: Comparative analysis of defect resolution intensity

Approach	SSE	Runtime
Resolution process ($m_r^b(t)$)	3.77×10^4	175.2
Resolution process ($m_r^d(t)$)	2.96×10^5	1627.2
Distributional	$\mathbf{3.43 \times 10^4}$	**150.1**
Markovian	3.88×10^4	298.8

The model with the lowest SSE and runtime, namely the distributional approach, are shown in bold.

The maximum likelihood estimates of $m_r^b(t)$ were $\hat{a} = 465.13$, $\hat{b} = 0.004855$, and $\hat{c} = 101.605$, while the maximum likelihood estimates of $m_r^d(t)$ were $\hat{a} = 462.617$, $\hat{b} = 0.004191$, $\hat{c} = 167.91$, and $\hat{d} = 0.656773$. Thus, the unique resolution rate d in Equation (6) contributes to the poor quality of the model fit because of the higher masking term c, which corresponds to visible underprediction in the number of defects resolved in the interval $t = (600, 1000)$ of Figure 4.

The mean time from defect discovery to resolution of the distributional approach was $E[T_r] = 59.74$, which was determined with the special case of Equation (7), where all data up to the time of the resolution of the nth defect was available and the corresponding estimate for the Markovian approach from Equation (11) was $E[T_r] = 81.48$. Given that the Markovian approach required twice as much time to compute and still exhibited lower accuracy, this level of complexity may not be justified despite the additional process details underlying the model. Moreover, the accuracy and efficiency of the distributional approach may offer a competitive alternative to the resolution process ($m_r^b(t)$) of Equation (5).

2) Distributional approach: Seventeen possible distributions were considered, explaining why the calculation in Table III required 150 seconds. These distributions included the Beta, Birnbaum-Saunders, Exponential, Extreme value, Gamma, Generalized extreme value, Generalized Pareto, Inverse Gaussian, Logistic, Log-logistic, Lognormal, Nakagami, Normal, Rayleigh, Rician, t location-scale, and Weibull distributions. The generalized extreme value (GEV) distribution possessing the following maximum likelihood estimates $T_r \sim GEV(\mu = 57.4437, \sigma = 22.6722, \xi = -0.0959)$ attained the best fit to the empirical times between defect discovery and resolution according to AIC [30] and BIC [31] measurements, as shown in Figure 5.

However, this retroactive approach is only useful for model assessment after all of the data has been collected and remains a pervasive problem in the defect discovery modeling literature, namely the over reliance on in sample measures of goodness of fit and the lack of online procedures that can be employed during the defect discovery and resolution process, which are explicitly addressed in Section IV-E.

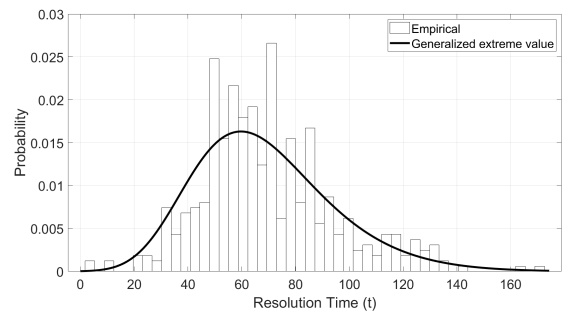

Fig. 5: Empirical distribution of time between defect discovery and resolution

3) Markovian approach: The defect tracking state model shown in Figure 2 contains 26 distinct transitions, each possessing a unique distribution. Since the path each defect takes through the state model can differ, some of these transitions were observed infrequently. In cases where five or fewer transitions were recorded the mean time between transitions was computed according to the arithmetic mean. Table IV reports the 16 transitions between states i and j with greater than five observations, the distribution that attained the best fit, sample size ($n_{i,j}$), and parameters of the model of best fit.

TABLE IV: Empirical state transition distributions

Transition		Distribution	$n_{i,j}$	MLE		
i	j			Shape	Scale	Location
0	1	Generalized Pareto	62	0.36	7.39	0.08
0	3	Log-logistic	8		0.94	2.16
0	7	Birnbaum-Saunders	12	2.34	5.70	
0	8	Exponential	6		10.22	
1	0	Rician	13		0.14	0.57
1	3	Exponential	6		71.42	
1	7	Birnbaum-Saunders	55	1.91	12.78	
2	9	Generalized Pareto	33	0.79	10.77	11.29
2	10	Birnbaum-Saunders	31	1.39	27.98	
6	11	Nakagami	20	63723.31	0.46	
7	12	Generalized Pareto	65	0.89	9.34	0.99
8	3	Exponential	6		341.94	
10	6	Weibull	21	0.79	61.89	
10	9	Birnbaum-Saunders	9	1.42	63.17	
11	9	Generalized Pareto	20	0.42	29.69	0.99
12	2	Generalized Pareto	65	0.23	45.44	0.99

The sample sizes reported in Table IV were used to estimate the transition probabilities $q_{i,j}$. For example, $\hat{q}_{0,1} = \frac{62}{88} = 0.7045$ is the fraction of transitions from the *Created (0)* to *Assigned (1)* state. The expected number of visits to each state was then calculated with Equation (10). The mean time to transition between states $E[T_{i,j}]$ was then calculated according to the formula for the mean of the distribution of best fit and corresponding numerical parameters or arithmetic mean. Finally, the average time from defect discovery to resolution was computed according to Equation (11).

D. Defect resolution intensity

This section performs a comparative analysis of the resolution intensity functions of each model. We begin with an explanation of how the empirical intensity was computed.

Authorized licensed use limited to: University of Massachusetts - Dartmouth. Downloaded on April 09,2021 at 16:01:02 UTC from IEEE Xplore. Restrictions apply.

In some cases, the fidelity of the defect tracking database was limited, containing instances where two or more successive defects were resolved at the exact same time. This led to points at which the empirical defect resolution intensity was infinity, since the reciprocal of the time between resolutions was $\frac{1}{0}$. To enable calculation of the empirical defect resolution intensity despite this limitation, the following more general expression was employed

$$\lambda_r(t_{r_i}) = \frac{N_r(t_{r_i}) - N_r(t_{r_{i-1}})}{\sum_{j \in R} t_{r_i} - t_j} \tag{17}$$

where $N_r(t_{r_i}) - N_r(t_{r_{i-1}})$ is the number of defects resolved at the ith unique resolution time t_{r_i}, R the set of defects resolved (since defects are not necessarily resolved in the order they were discovered), and $t_{r_i} - t_j$ the time between the discovery of defect $j \in R$ which was simultaneously resolved at t_{r_i}. Thus, the denominator is the total time the defects were worked on prior to their resolution, since defects were not necessarily all discovered at the same time.

Figure 6 shows the step function associated with the empirical resolution intensity computed at each resolution time t_{r_i} along with the resolution intensity of each fitted model, computed by differentiating the mean value of function of the defect resolution counting process with respect to t and substituting the MLEs used to plot Figure 4 into these expressions.

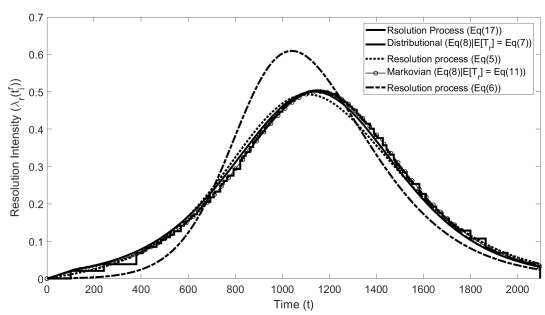

Fig. 6: Defect resolution intensity

With the exception of the model possessing unique resolution rate d of Equation (6), the models track the empirical intensity well.

To compare the models quantitatively, Table V summarizes the sum of squares error between the empirical resolution intensity and fitted curves.

TABLE V: Comparative analysis of defect resolution intensity

Approach	SSE
Resolution process ($m_r^b(t)$)	5.627×10^3
Resolution process ($m_r^d(t)$)	5.665×10^4
Distributional	5.595×10^3
Markovian	$\mathbf{5.486 \times 10^3}$

The results indicate that the Markovian approach performed best, followed closely by the distributional approach and simpler resolution process possessing intensity ($\lambda_r^b(t)$).

E. Comparison of online predictive accuracy and computational efficiency

To further compare the alternative defect resolution models, this section conducts an online assessment of predictive accuracy, which also explicitly considers the computational efficiency of each approach. Starting at time $t = 200$, the defect resolution processes of Equations (5) and (6) were fit directly to the resolution time data extracted from the defect tracking database, while the distributional and Markovian approaches first identified an SRGM that best fit the available defect detection data,, and then estimated the mean time to restore defects ($E[T_r]$) with Equations (7) and (11) respectively and substituted this value into Equation (8). In this manner each model, was used to predict the cumulative number of defects resolved 200 time units into the future or $t = 400$ and the percentage error computed between the actual number of defects observed and the predicted values. This process was repeated periodically for each model. However, Table III determined that the model with unique defect resolution rate ($m_r^d(t)$) was over an order of magnitude slower than the distributional approach. Thus, the frequency of predictions made by each model was chosen to be inversely portion to their runtime. This method was implemented to allocate approximately equal computational time to each method.

Figure 7 shows the results of online assessment of predictive accuracy explicitly considering computational efficiency.

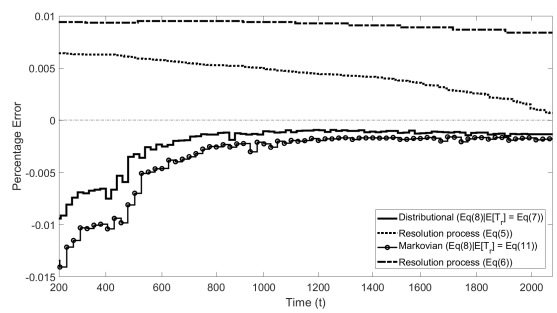

Fig. 7: Comparison of error predicted by distributional and Markovian resolution time models

Figure 7 indicates that the distributional approach achieves the lowest error as early as $t = 300$ and remains the most accurate throughout the remainder of the defect testing and resolution process. Moreover, the computational efficiency enables frequent updates, supporting regular online assessment throughout. Unlike the defect resolution processes, which rely solely on the defect resolution count data, the distributional approach explicitly incorporates information on defects discovered and resolved as well as those discovered but not yet resolved, whereas the Markovian approach provides even greater fidelity, including rows associated with defects that

Authorized licensed use limited to: University of Massachusetts - Dartmouth. Downloaded on April 09,2021 at 16:01:02 UTC from IEEE Xplore. Restrictions apply.

traversed from *Start* to *End* in Figure 2 as well as defects that have only partially traversed the state model in Figure 2.

V. Conclusion

This paper connect software reliability growth models to defect tracking databases. Differential-equation based, distributional, and Markovian approaches were developed. A NASA defect tracking database was cleaned and a state model constructed. A statistical hypothesis test was applied to determine the order if a first or second order Markov chain best characterized the transitions taken during the defect lifecycle. Models were applied to the full data, referred to as retrospective analysis, and the steps taken to apply the distributional and Markovian approaches explained. The goodness of fit of alternative models was also assessed with respect to the defect resolution intensity. Finally, the predictive accuracy and computational efficiency were assessed in an online manner. The results suggested that most of the models fit the defect resolution data well, but that the distributional approach achieved the best fit and also demonstrated the lowest runtime, enabling more frequent updates, which would support online tracking when defect detection and resolution are ongoing. Despite the deficiencies in the data, which posed the most obvious threat to validity, the models demonstrated the potential benefit of renewed efforts to collect high quality data related to the defect tracking lifecycle.

Some useful directions to explore in the future include: (i) application of covariate models to determine the relative effectiveness of alternative activities employed to resolve defects and identify how to allocate resources to these activities efficiently and (ii) identify factors that could be used to modeling the cost to resolve defects to inform resource planning.

Acknowledgment

This work was supported by the National Aeronautics and Space Administration (NASA) under Grant Number (#80NSSC18K0154) and NSF award (#1749635). Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the sponsors.

References

[1] W. Farr, *Handbook Of Software Reliability Engineering*. New York, NY: McGraw-Hill, 1996, chapter Software Reliability Modeling Survey, pp. 71–117.

[2] N. Schneidewind, "Analysis of error processes in computer software," in *Proc. ACM Sigplan Notices*, vol. 10, no. 6, 1975, pp. 337–346.

[3] M. Xie and M. Zhao, "The Schneidewind software reliability model revisited," in *Proc. IEEE International Symposium on Software Reliability Engineering*, 1992, pp. 184–192.

[4] M. Ohba, "Inflection S-shaped software reliability growth model," in *Stochastic models in reliability theory*. Springer, 1984, pp. 144–162.

[5] S. Yamada, M. Ohba, and S. Osaki, "S-shaped reliability growth modeling for software error detection," *IEEE Transactions on Reliability*, vol. 32, no. 5, pp. 475–484, 1983.

[6] P. Kapur and S. Younes, "Software reliability growth model with error dependency," *Microelectronics Reliability*, vol. 35, no. 2, pp. 273–278, 1995.

[7] S. Yamada, K. Tokuno, and S. Osaki, "Imperfect debugging models with fault introduction rate for software reliability assessment," *International Journal of Systems Science*, vol. 23, no. 12, pp. 2241–2252, 1992.

[8] S. Gokhale, P. Marinos, M. Lyn, and K. Trivedi, "Effect of repair policies on software reliability," in *Proc. IEEE Annual Conference on Computer Assurance*, 1997, pp. 105–116.

[9] S. Gokhale and R. Mullen, "Queuing models for field defect resolution process," in *Proc. IEEE International Symposium on Software Reliability Engineering*, 2006, pp. 353–362.

[10] K. Shibata, K. Rinsaka, T. Dohi, and H. Okamura, "Quantifying software maintainability based on a fault-detection/correction model," in *Proc. IEEE Pacific Rim International Symposium on Dependable Computing*, 2007, pp. 35–42.

[11] C.-Y. Huang, C.-T. Lin, S.-Y. Kuo, M. Lyu, and C. Sue, "Software reliability growth models incorporating fault dependency with various debugging time lags," in *Proc. IEEE International Computer Software and Applications Conference*, 2004, pp. 186–191.

[12] J.-H. Lo and C.-Y. Huang, "An integration of fault detection and correction processes in software reliability analysis," *Journal of Systems and Software*, vol. 79, no. 9, pp. 1312–1323, 2006.

[13] N. Ullah, M. Morisio, and A. Vetro, "A comparative analysis of software reliability growth models using defects data of closed and open source software," in *Proc. IEEE Software Engineering Workshop*, 2012, pp. 187–192.

[14] Y. Liu, M. Xie, J. Yang, and M. Zhao, "A new framework and application of software reliability estimation based on fault detection and correction processes," in *Proc. IEEE International Conference on Software Quality, Reliability and Security*, 2015, pp. 65–74.

[15] J. Yang, Y. Liu, M. Xie, and M. Zhao, "Modeling and analysis of reliability of multi-release open source software incorporating both fault detection and correction processes," *Journal of Systems and Software*, vol. 115, pp. 102 – 110, 2016.

[16] M. Cinque, D. Cotroneo, A. Pecchia, R. Pietrantuono, and S. Russo, "Debugging-workflow-aware software reliability growth analysis," *Software Testing, Verification and Reliability*, vol. 27, no. 7, p. e1638, 2017.

[17] H. Okamura and T. Dohi, "A generalized bivariate modeling framework of fault detection and correction processes," in *Proc. IEEE International Symposium on Software Reliability Engineering*, 2017, pp. 35–45.

[18] P. Vizarreta, K. Trivedi, B. Helvik, P. Heegaard, A. Blenk, W. Kellerer, and C. M. Machuca, "Assessing the maturity of sdn controllers with software reliability growth models," *IEEE Transactions on Network and Service Management*, vol. 15, no. 3, pp. 1090–1104, 2018.

[19] P. Vizarreta, E. Sakic, W. Kellerer, and C. M. Machuca, "Mining software repositories for predictive modelling of defects in sdn controller," in *Proc. IFIP/IEEE Symposium on Integrated Network and Service Management*, 2019, pp. 80–88.

[20] H. Sukhwani, J. Alonso, K. Trivedi, and I. Mcginnis, "Software reliability analysis of NASA space flight software: A practical experience," in *Proc. IEEE International Conference on Software Quality, Reliability and Security*, 2016, pp. 386–397.

[21] A. Goel and K. Okumoto, "Time-dependent error-detection rate model for software reliability and other performance measures," *IEEE Transactions on Reliability*, vol. 28, no. 3, pp. 206–211, aug 1979.

[22] S. Yamada, H. Ohtera, and H. Narihisa, "Software reliability growth models with testing-effort," *IEEE Transactions on Reliability*, vol. 35, no. 1, pp. 19–23, 1986.

[23] S. Yamada, J. Hishitani, and S. Osaki, "Software-reliability growth with a Weibull test-effort: A model & application," *IEEE Transactions on Reliability*, vol. 42, no. 1, pp. 100–106, mar 1993.

[24] Z. Jelinski and P. Moranda, "Software reliability research," *Statistical computer performance evaluation*, pp. 465–484, 1972.

[25] P. Moranda, "Prediction of software reliability during debugging," in *Proc. Annual Reliability and Maintainability Symposium*, 1975.

[26] I. Gertsbakh, *Reliability Theory: With Applications to Preventive Maintenance*. Berlin: Springer, 2000.

[27] S. Karlin and H. E. Taylor, *A Second Course in Stochastic Processes*. New York, NY: Elsevier, 1981.

[28] A. Berchtold and A. E. Raftery, "The mixture transition distribution model for high-order Markov chains and non-Gaussian time series," *Statistical Science*, pp. 328–356, 2002.

[29] M. Tainiter, "Estimation, hypothesis testing and parameter correlation for Markov chains," *IEEE Transactions on Reliability*, vol. 12, no. 4, pp. 26–35, 1963.

[30] H. Akaike, "A new look at the statistical model identification," *IEEE Transactions on Automatic Control*, vol. 19, no. 6, pp. 716–723, 1974.

[31] G. Schwarz, "Estimating the dimension of a model," *The Annals of Statistics*, vol. 6, no. 2, pp. 461–464, 1978.

Authorized licensed use limited to: University of Massachusetts - Dartmouth. Downloaded on April 09,2021 at 16:01:02 UTC from IEEE Xplore. Restrictions apply.